COMPTIA SECURITY+ SY0-701 | DOMÍNIO 2 · AMEAÇAS, VULNERABILIDADES E MITIGAÇÕES (22%)

Security+ 2.4 · Replay Attacks

Mestre CyberSec · **Aula 2.4 — Replay Attacks** (5:45) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: Replay: capturar o handshake/token e REENVIAR pra se autenticar de novo. Pass-the-hash: usar o HASH da senha como autenticador direto NTLM (Mimikatz + PsExec) — movimento lateral. Session hijacking: roubar cookie de sessão ativa (via XSS ou on-path) e se passar pelo usuário. Defe

💡 Highlights que a prova CompTIA cobra

- Replay: reenvia o 'handshake' capturado — defesa: nonce/timestamp
- Pass-the-hash: usa O HASH (não a senha) — NTLM é vulnerável, Kerberos menos
- Session hijacking: rouba cookie — HttpOnly + HTTPS mitiga

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'reuses captured authentication data' = replay attack
- 'uses an NTLM hash to authenticate' = pass-the-hash

🚨 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 2.4 — Replay Attacks)

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Ataque de Replay

Ataque que captura dados de autenticação válidos, como um handshake ou token, e os reenvia para se autenticar como o usuário original. Não quebra a criptografia; reutiliza credenciais capturadas.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'reuses captured authentication data' = replay attack. Defesa principal: nonce e timestamp para tornar o dado único e não reutilizável.*

• Pass-the-Hash (PtH)

Ataque que usa um hash de senha NTLM capturado como autenticador direto, sem quebrá-lo para texto claro. Comumente usa Mimikatz para extrair hashes e PsExec para movimento lateral.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'uses an NTLM hash to authenticate' = pass-the-hash. NTLM é vulnerável; Kerberos reduz esse risco.*

• Sequestro de Sessão

Ataque que rouba um cookie ou token de sessão ativo, frequentemente via XSS ou interceptação on-path, e o usa para se passar pelo usuário autenticado.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'steals session cookie' = session hijacking. Mitigações: HttpOnly e HTTPS impedem acesso via script e protegem o cookie em trânsito.*

• Nonce

Valor aleatório ou único usado uma vez em uma troca de autenticação para garantir atualidade e impedir replay de mensagens capturadas.

⚠️ ***Armadilha CompTIA:*** *Defesa contra replay: nonce torna cada handshake único. Se a prova citar 'number used once', é nonce.*

• Carimbo de Tempo

Valor de tempo incluído nos dados de autenticação para verificar atualidade e rejeitar mensagens antigas demais para serem válidas.

⚠️ ***Armadilha CompTIA:*** *Defesa contra replay: se o timestamp estiver fora da janela aceitável, a autenticação é negada. Distinga de nonce: tempo vs. valor único.*

• **NTLM**
Protocolo de autenticação desafio-resposta do Windows que pode ser abusado no pass-the-hash porque a posse do hash da senha é suficiente para autenticar.

⚠️ ***Armadilha CompTIA:*** *Gatilho: NTLM + hash = pass-the-hash. Kerberos é preferível e menos suscetível a esse tipo de ataque.*

• **Kerberos**
Protocolo de autenticação baseado em tickets que é menos vulnerável ao pass-the-hash que o NTLM por não depender do envio de hashes de senha para autenticar.

⚠️ ***Armadilha CompTIA:*** *Distinga: NTLM é associado a pass-the-hash; Kerberos usa tickets e é a alternativa mais segura em domínios Windows.*

• **HttpOnly**
Atributo de cookie que impede scripts do lado do cliente, como JavaScript, de acessar o cookie, reduzindo roubo de sessão via XSS.

⚠️ ***Armadilha CompTIA:*** *Mitigação de session hijacking: HttpOnly protege contra XSS; HTTPS protege o cookie em trânsito. Se a questão citar 'script access to cookie', responda HttpOnly.*

🏛️ Visão Didática, Analogias & Análise de Governança (GRC)

📌 SSRF (Server-Side Request Forgery)

🏠 **Analogia Intuitiva:** É como você ir até o guichê de atendimento de um prédio governamental ultra-protegido e entregar um bilhete para o mensageiro oficial dizendo: 'Por favor, vá até a sala secreta dos arquivos presidenciais no 10º andar, tire uma foto do documento confidencial e traga aqui para eu ver na recepção'. Como o mensageiro é um funcionário credenciado da casa que tem crachá de livre acesso a todos os setores internos (onde você, como cidadão comum da rua, é barrado pelos seguranças), ele obedece sem desconfiar, busca o segredo confidencial na rede interna restrita e entrega para você no balcão da calçada.

🔍 **Causa Raiz / Por que foi criado:** Aplicações web frequentemente precisam buscar recursos externos (como importar imagens via URL ou consultar webhooks). Se a aplicação não validar rigorosamente o endereço de destino fornecido pelo usuário, o invasor submete URLs apontando para a rede interna ou loopback (`http://127.0.0.1`, `http://localhost`) ou para o serviço de metadados da nuvem (`http://169.254.169.254/latest/meta-data/`), roubando credenciais de IAM e segredos corporativos.

🛡️ **Visão do Auditor de GRC:** Auditar a aplicação de listas de permissão (allowlists) de domínios externos; desabilitar resolução de esquemas não web (como `file://` ou `gopher://`); bloquear resoluções para faixas IP privadas na camada de aplicação; e exigir a ativação do IMDSv2 em instâncias de nuvem (que exige cabeçalho de token de sessão HTTP PUT para neutralizar SSRF).

⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker tricks a public web server into executing requests against internal network resources or the cloud metadata endpoint 169.254.169.254' -> SSRF (Server-Side Request Forgery).

📌 CSRF (Cross-Site Request Forgery)

🏠 **Analogia Intuitiva:** Você já está com a sua sessão aberta no site do banco no navegador. Em outra aba, você clica em um link de meme ou promoção falsa que um desconhecido te enviou. A página fraudulenta contém um código invisível que manda um pedido para o seu banco: 'Transfira R$ 2.000 para a conta do golpista'. O seu navegador envia o pedido anexando automaticamente os seus cookies de login legítimos. Como o banco reconhece seu cookie de sessão autêntico, ele acha que foi você quem ordenou a transferência e autoriza a operação sem você saber.

🔍 **Causa Raiz / Por que foi criado:** Navegadores web automaticamente incluem cookies armazenados de autenticação em todas as requisições enviadas ao domínio de destino, mesmo quando o disparo é originado por páginas de terceiros. Sem validação de intenção através de tokens exclusivos por transação, o servidor não consegue distinguir requisições voluntárias de requisições forjadas.

🛡️ **Visão do Auditor de GRC:** Implementar tokens anti-CSRF aleatórios e imprevisíveis (Synchronizer Token Pattern) em formulários com alteração de estado; configurar o atributo `SameSite=Strict` ou `SameSite=Lax` em todos os cookies de sessão; e exigir reautenticação para transações financeiras críticas.

> ⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker tricks a victim's authenticated browser into executing unwanted actions on a trusted web application' -> Cross-Site Request Forgery (CSRF). Mitigações padrão: Anti-CSRF Tokens e atributo de cookie SameSite.

📌 BOLA (Broken Object Level Authorization) e Falhas de API

🏠 **Analogia Intuitiva:** BOLA (antigo IDOR) é como você ir ao armário de guarda-volumes da academia com a chave do armário número 101. Ao terminar o treino, você olha para o armário do lado, vê a fechadura do armário 102 e simplesmente gira a maçaneta ou usa um clipe de papel; a porta do armário 102 se abre imediatamente e você vê as roupas e carteira de outro aluno. O sistema conferiu que você era um aluno matriculado da academia (autenticação válida), mas não checou se você tinha autorização para abrir ESPECIFICAMENTE o armário 102.

🔍 **Causa Raiz / Por que foi criado:** Em APIs REST modernas, o servidor recebe requisições com IDs de objetos fornecidos pelo cliente (ex: `/api/v1/invoices?id=5002`). A aplicação confere se o token de acesso do usuário é válido (autenticação), mas não valida se o usuário autenticado possui o direito de posse/leitura daquele objeto específico 5002 (autorização a nível de objeto). BOLA é a falha nº 1 do OWASP API Top 10.

🛡️ **Visão do Auditor de GRC:** Implementar controle de acesso baseado em atributos (ABAC) validando a posse do objeto no banco de dados para cada requisição; substituir IDs numéricos incrementais sequenciais por identificadores opacos e imprevisíveis (UUIDv4); e utilizar API Gateways para fiscalização de esquemas de autorização.

> ⚠️ **Como cai na Prova CompTIA:** Trigger: 'user modifies an account number or invoice ID in an API request (e.g., from id=101 to id=102) and successfully accesses another client's records'. Resposta: Broken Object Level Authorization (BOLA) ou IDOR.

📌 Poluição de Parâmetros HTTP (HTTP Parameter Pollution - HPP)

🏠 **Analogia Intuitiva:** É como preencher um formulário impresso de pagamento que tem o campo 'Favorecido:' e você escrever duas vezes no formulário: na primeira linha você escreve o nome da sua empresa e na segunda linha escreve o nome de um criminoso. O auditor do portão de entrada (WAF) olha apenas para a primeira linha, acha legal e aprova; mas o caixa do cofre no porão (servidor backend) olha apenas para a segunda linha e transfere todo o dinheiro para o criminoso. A divergência de interpretação entre quem fiscaliza e quem executa cria a brecha.

🔍 **Causa Raiz / Por que foi criado:** Diferentes servidores e tecnologias web lidam com parâmetros duplicados de maneiras conflitantes (alguns consideram o primeiro, outros consideram o último, outros concatenam com vírgula). Invasores exploram essa discrepância enviando requisições com parâmetros repetidos (`?user=guest&user=admin`) para burlar filtros de segurança do WAF e escalar privilégios no backend.

🛡️ **Visão do Auditor de GRC:** Padronizar o tratamento de parâmetros em toda a pilha arquitetural; rejeitar requisições HTTP que contenham nomes de parâmetros duplicados no cabeçalho ou corpo; e validar entradas contra especificações OpenAPI estritas.

> ⚠️ **Como cai na Prova CompTIA:** Trigger: 'attacker supplies multiple HTTP parameters with the same name to bypass perimeter validation filters due to differences in parsing logic between proxy and application' -> HTTP Parameter Pollution (HPP).

⚡ TABELA DE FIXAÇÃO RÁPIDA PARA O EXAME (CHEAT SHEET)

Cenário Típico de Exame / Situação Prática	Conceito Técnico & Resposta da Banca
Atacante induz o servidor web a realizar requisições não autorizadas para recursos internos ou para o IP de metadados da nuvem (169.254.169.254)	**Server-Side Request Forgery (SSRF)**
Página maliciosa de terceiros induz o navegador autenticado do usuário a submeter transações indesejadas em uma aplicação confiável	**Cross-Site Request Forgery (CSRF)**
Usuário autenticado altera o parâmetro de ID em uma chamada de API (ex: id=101 para 102) e visualiza dados privados de outro cliente	**Broken Object Level Authorization (BOLA) / IDOR**
Atacante envia múltiplos parâmetros com o mesmo nome para explorar divergências de interpretação entre o WAF e o backend	**Poluição de Parâmetros HTTP (Parameter Pollution / HPP)**
Defesa web que anexa um token criptográfico descartável a cada formulário para impedir que ações forjadas sejam aceitas	**Token Anti-CSRF (Synchronizer Token)**
Versão do serviço de metadados da AWS que exige cabeçalho de token de sessão HTTP PUT, mitigando ataques de SSRF	**IMDSv2 (Instance Metadata Service Version 2)**

✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Uma aplicação web financeira aceita requisições de API assinadas para transferências de contas. Um testador de penetração grava uma requisição de transferência legítima e a envia novamente mais tarde, sem alterá-la. O servidor atualmente verifica apenas a assinatura digital. Qual controle deve ser adicionado para impedir esse ataque?

A) Calcular hash do corpo da requisição com SHA-256 antes de assiná-lo.

B) Incluir um valor imprevisível de uso único em cada requisição e rejeitar qualquer valor reutilizado.

C) Exigir TLS 1.3 para todas as conexões de API.

D) Forçar os usuários a trocar senhas a cada 90 dias.

QUESTÃO 2 · CENÁRIO DE EXAME

Durante um incidente, analistas descobrem que um atacante obteve material de credencial de uma estação de trabalho e o usou para se autenticar em vários servidores de arquivos. O atacante nunca quebrou a senha e nunca soube o valor em texto claro. Qual técnica de ataque isso descreve?

A) Session hijacking

B) Kerberoasting

C) Pass-the-hash

D) Ataque de replay

QUESTÃO 3 · CENÁRIO DE EXAME

Uma aplicação web armazena identificadores de sessão em cookies. Um testador de segurança demonstra que JavaScript injetado consegue ler o cookie de sessão e enviá-lo a um servidor externo. Qual atributo de cookie impede mais diretamente esse roubo específico baseado em script?

A) Secure

B) SameSite=Strict

C) HttpOnly

D) Domain

QUESTÃO 4 · CENÁRIO DE EXAME

Funcionários usam uma rede Wi-Fi pública. Um atacante na mesma rede captura um cookie de sessão autenticada enviado por HTTP e o usa para acessar a conta do funcionário. Qual mitigação aborda melhor esse risco?

A) Definir cookies de sessão com o sinalizador Secure e impor HTTPS com HSTS.

B) Armazenar identificadores de sessão no armazenamento local do navegador e validá-los apenas no lado do cliente.

C) Reduzir os tempos limite de sessão para cinco minutos.

D) Habilitar compressão e cache HTTP para a aplicação.

QUESTÃO 5 · CENÁRIO DE EXAME

Um domínio Windows usa autenticação Kerberos. Arquitetos de segurança querem garantir que um atacante que capture uma troca de autenticação válida não consiga simplesmente retransmiti-la mais tarde. Em qual mecanismo o Kerberos se baseia para limitar esse tipo de reutilização?

A) Um cache de hash NTLM

B) Um autenticador com carimbo de data/hora

C) Um cookie de sessão com SameSite

D) Um ticket de serviço permanente

QUESTÃO 6 · CENÁRIO DE EXAME

Uma organização quer reduzir o risco de um atacante que compromete uma estação de trabalho Windows usar material de credencial em cache para se autenticar em outros sistemas como administrador. Qual controle é mais eficaz?

A) Habilitar os sinalizadores HttpOnly e Secure em cookies do navegador.

B) Implantar o Credential Guard e restringir contas privilegiadas a sistemas de administração dedicados.

C) Implementar CAPTCHA na página de login da VPN.

D) Exigir criptografia AES para todas as contas de serviço Kerberos.

QUESTÃO 7 · CENÁRIO DE EXAME

Um protocolo de autenticação personalizado envia ao servidor uma resposta derivada da senha. Um atacante captura a resposta e se autentica com sucesso enviando exatamente a mesma resposta novamente. Qual mudança de projeto impede melhor isso?

A) Aumentar o comprimento mínimo da senha.

B) Codificar a resposta em Base64 antes da transmissão.

C) Usar um desafio aleatório gerado pelo servidor em cada tentativa de autenticação e incluí-lo na resposta.

D) Armazenar toda resposta derivada de senha em um banco de dados para auditoria.

QUESTÃO 8 · CENÁRIO DE EXAME

Uma aplicação web atribui um ID de sessão quando o usuário visita o site pela primeira vez e mantém o mesmo ID após o login. Um atacante pode definir um ID de sessão conhecido no navegador da vítima e usá-lo depois que a vítima se autenticar. Qual correção aborda melhor essa vulnerabilidade?

A) Marcar o cookie de sessão como HttpOnly.

B) Adicionar um nonce a cada requisição de login.

C) Regenerar o ID de sessão imediatamente após a autenticação bem-sucedida.

D) Habilitar autenticação NTLM para o servidor web.

QUESTÃO 9 · CENÁRIO DE EXAME

Alertas do SIEM mostram múltiplas autenticações NTLM bem-sucedidas de uma única conta de usuário para muitos servidores. A estação do usuário não registra entrada interativa de senha, e o processo de origem não é um navegador. Qual atividade é mais consistente com esses indicadores?

A) Cross-site scripting roubando cookies

B) Pass-the-hash usando um hash de credencial roubado

C) Replay de ticket Kerberos

D) Fixação de sessão

QUESTÃO 10 · CENÁRIO DE EXAME

Um sistema de autenticação multifator aceita um código de uso único de seis dígitos por cinco minutos. Um atacante intercepta um código válido e o usa dentro da janela permitida. Qual melhoria impede melhor a reutilização do mesmo código?

A) Aumentar o comprimento do código para oito dígitos.

B) Trocar o aplicativo autenticador por entrega via SMS.

C) Registrar cada código como usado e vinculá-lo à sessão ou desafio de autenticação específico.

D) Armazenar códigos emitidos no armazenamento local do navegador para validação mais rápida.

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa B — Incluir um valor imprevisível de uso único em cada requisição e rejeitar qualquer valor reutilizado.

Uma assinatura digital prova integridade e origem, mas não impede que um atacante reenvie uma requisição válida capturada. Um nonce é um valor único e imprevisível usado uma vez; o servidor rastreia nonces usados e rejeita duplicatas. Hash, TLS e rota��ão de senha não impedem replay na camada de aplicação.

Questão 2: Alternativa C — Pass-the-hash

Pass-the-hash ocorre quando um atacante captura um hash de senha NTLM e usa esse hash como autenticador, sem quebrá-lo para texto claro. Session hijacking usa um token ou cookie de sessão roubado, Kerberoasting visa tickets de contas de serviço Kerberos, e replay envolve retransmitir uma mensagem de autenticação capturada.

Questão 3: Alternativa C — HttpOnly

HttpOnly impede que scripts do lado do cliente leiam o cookie, bloqueando o roubo via XSS. Secure garante que o cookie só seja enviado por HTTPS, SameSite limita o envio entre sites e Domain define onde o cookie é válido; nenhum deles impede diretamente o acesso por JavaScript.

Questão 4: Alternativa A — Definir cookies de sessão com o sinalizador Secure e impor HTTPS com HSTS.

O problema central é que o cookie de sessão é transmitido sem criptografia e pode ser capturado no caminho. Cookies Secure só são enviados por HTTPS, e HSTS impõe conexões HTTPS, evitando exposição em texto claro. Armazenamento no cliente, timeouts curtos e compressão/cache não impedem roubo de cookie on-path.

Questão 5: Alternativa B — Um autenticador com carimbo de data/hora

O Kerberos usa autenticadores que incluem um carimbo de data/hora e são criptografados com uma chave de sessão. O servidor verifica o carimbo para rejeitar autenticadores antigos ou reutilizados. Cache de hash NTLM está relacionado a Pass-the-Hash, cookies são tokens de sessão web, e tickets de serviço permanentes aumentariam o risco de replay em vez de impedi-lo.

Questão 6: Alternativa B — Implantar o Credential Guard e restringir contas privilegiadas a sistemas de administração dedicados.

O Credential Guard ajuda a isolar e proteger hashes de senha NTLM e outros segredos contra roubo, reduzindo o risco de Pass-the-Hash. Restringir contas privilegiadas a sistemas de administração dedicados limita o movimento lateral. Sinalizadores de cookie, CAPTCHA e criptografia Kerberos para contas de serviço não protegem diretamente hashes de credenciais em cache na estação.

Questão 7: Alternativa C — Usar um desafio aleatório gerado pelo servidor em cada tentativa de autenticação e incluí-lo na resposta.

Um desafio aleatório gerado pelo servidor, ou nonce, torna cada troca de autenticação única. Reutilizar uma resposta capturada não corresponderá ao novo desafio. Senhas mais longas, codificação Base64 e registro de respostas não impedem replay do valor de autenticação capturado.

Questão 8: Alternativa C — Regenerar o ID de sessão imediatamente após a autenticação bem-sucedida.

Isso descreve fixação de sessão: o atacante fixa um ID de sessão conhecido antes do login e o reutiliza depois. Regenerar o ID de sessão após a autenticação invalida o ID conhecido pelo atacante. HttpOnly ajuda contra roubo por XSS, mas não contra fixação, nonces de login tratam replay, e NTLM não se relaciona ao gerenciamento de sessão web.

Questão 9: Alternativa B — Pass-the-hash usando um hash de credencial roubado

Autenticações NTLM bem-sucedidas para muitos servidores sem entrada interativa de senha são um indicador comum de Pass-the-Hash, especialmente quando uma ferramenta usa um hash roubado como credencial. Roubo de cookie por XSS, replay de ticket Kerberos e fixação de sessão envolveriam cookies web ou artefatos Kerberos, não esse padrão de autenticação NTLM.

Questão 10: Alternativa C — Registrar cada código como usado e vinculá-lo à sessão ou desafio de autenticação específico.

Interceptar e reutilizar um OTP válido dentro da janela de validade é um ataque de replay. Marcar OTPs como usados e vinculá-los a uma sessão ou desafio impede que o mesmo código seja aceito duas vezes. Códigos mais longos, SMS e armazenamento no cliente não impedem replay e podem enfraquecer a segurança.